

Unit 1: Introduction to Software Engineering

Software, in its most general sense, is a set of instructions or programs instructing a computer to do specific tasks.

Software is more than just a program code. A program is an executable code, which serves some computational purpose. Software is considered to be a collection of executable programming code, associated libraries and documentations.

Software Characteristics

Software Characteristics are classified into six major components:



Definition of Software Engineering

The term ***software engineering*** is composed of two words, software and engineering.

Software is more than just a program code. A program is an executable code, which serves some computational purpose. Software is considered to be a collection of executable programming code, associated libraries and documentations. Software, when made for a specific requirement is called **software product**.

Engineering on the other hand, is all about developing products, using well-defined, scientific principles and methods. So, we can

define

software engineering as an engineering branch associated with the development of software product using well-defined scientific principles, methods and procedures. The outcome of software engineering is an efficient and reliable software product.

IEEE defines software engineering as:

The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software

1. Evolving Role of Software

Definition

Software is a collection of programs, data, and documents that instruct a computer to perform specific tasks. Over time, software has become one of the most important technologies in the world.

Earlier, software was used only for calculations and business records. Today, it is used in almost every area of life such as education, banking, healthcare, transportation, entertainment, communication, and scientific research.

Evolution of Software

1. Early Stage (1950–1970)

Software was developed only for specific computers.

Programs were small and simple.

Mostly used for mathematical calculations and scientific work.

Very few people used computers

2. Business Software Era

Companies started using software for payroll, accounting, inventory, and banking. Databases became popular.

Software helped automate business operations.

3. Personal Computer Era

Personal computers became common.

Office applications such as Word, Excel, and PowerPoint were introduced. Home users started using software for education and entertainment.

4. Internet Era

Websites became popular.

Email and online communication increased.

Online shopping and online banking started.

Social media applications became common.

5. Modern Era

Mobile applications are widely used.

Cloud computing allows software to run online.

Artificial Intelligence (AI) makes software smarter.

Current Role of Software

Software is used in almost every

field. **Education**

- Online classes

- Digital libraries
- Learning apps
- Online examinations

Healthcare

- Hospital management
- Patient records
- Medical diagnosis
- Online consultation

Banking

- Internet banking

- Mobile banking
- ATM systems
- Digital payments

Business

- Payroll management • Inventory management • Customer management •

Accounting

Entertainment

- Video streaming
- Music apps
- Online games

-

Transportation

- Online ticket booking
- GPS navigation
- Traffic management •

Flight reservation systems

Government

- Online services
- Digital identity systems

- Tax management
- E-governance

Importance of

Software • Saves time

- Reduces human effort
- Improves accuracy
- Increases productivity •

Stores large amounts of data •

Supports communication •

Automates complex tasks •

Helps in decision making

2. Changing Nature of Software

Definition

The changing nature of software means software technology keeps improving according to user needs and technological advancements.

Modern software is faster, smarter, more secure, and available on different devices. **Reasons for Change**

- New technologies
- Customer requirements

- Internet growth
- Mobile devices
- Cloud computing
- Artificial Intelligence
- Cybersecurity needs

Characteristics of Modern

Software 1. Web Applications

Software runs through internet browsers. Examples:

- Gmail

- Google Docs
- Amazon

Advantages

- No installation required
- Accessible from anywhere
- Easy updates

2. Mobile Applications

Applications designed for smartphones and tablets.

Examples

- WhatsApp
- Instagram
- Paytm

Features

- Portable
- Easy to use
- Fast communication

3. Cloud Computing

Software and data are stored on remote

servers.

Examples

- Google Drive
- Dropbox
- Microsoft OneDrive

Advantages

- Access from anywhere
- Automatic backup
- Low maintenance

4. Artificial Intelligence

AI allows software to think and learn. Examples

- Chatbots
- Voice assistants
- Recommendation systems

Benefits

- Automation

- Better decision making
- Personalized services

5. Internet of Things (IoT)

Physical devices communicate through software.

Examples

- Smart TV
- Smart Watch

- Smart Home
- Smart Car

6. Cyber Security

Software now includes strong security features.

Examples

- Password protection

- Encryption
- Firewalls
- Multi-factor authentication

Features of Modern

Software • User friendly

- Intelligent
- Secure
- Scalable

- Fast
- Cloud-enabled
- Mobile compatible
- Easy to maintain

Evolving Role and Changing Nature of Software

Software has evolved from simple programs used for basic calculations into an essential part of almost every field.

1. From standalone programs to complex systems: Earlier, software mainly performed specific tasks. Today, it includes large, interconnected systems such as cloud platforms, operating systems, and enterprise applications.

- 2. Integration into everyday life:** Software is now used in banking, healthcare, education, transportation, communication, entertainment, and many other areas.
- 3. Web and mobile applications:** The growth of the Internet and smartphones has changed software from desktop-based applications to web-based and mobile applications that can be accessed anywhere.
- 4. Cloud and AI-based software:** Modern software increasingly uses cloud computing, artificial intelligence, machine learning, and data analytics to provide intelligent and scalable service.

5. Changing nature of software: Software is no longer developed once and left unchanged. It is continuously updated, maintained, secured, and improved to meet changing user needs and technological requirements.

Thus, software has changed from a simple supporting tool into a critical technology that drives businesses, services, and modern society.

Software engineering – A layered technology

Software engineering is a fully layered technology. To develop software, we need to go from one layer to another. All the layers are connected

and each layer demands the fulfillment of the previous layer.



Layered technology is divided into four parts:

1. A quality focus

It defines the continuous process improvement principles of software. It provides integrity that means providing security to the software so that data can be accessed by only an authorized person, no outsider can access the data. It also focuses on maintainability and usability. It ensures

- .Reliable software
- .Secure
- .Correct output

- .High performance

- Customer satisfaction

2. Process

It is the foundation or base layer of software engineering. It is key that binds all the layers together which enables the development of software before the deadline or on time. Process defines a framework that must be established for the effective delivery of software engineering technology. The software process covers all the activities, actions, and tasks required to be carried out for software development.

Purpose

- .Organize work

- .Reduce errors
- .Improve quality
- .Complete projects on time

3. Method

During the process of software development the answers to all “how-to-do” questions are given by method. It has the information of all the tasks which includes communication, requirement analysis, design modelling, program construction, testing, and support.

They include

- .Requirement analysis

- .Software design
- .Coding
- .Testing
- .Maintenance

4. Tools

Software engineering tools provide a self-operating system for processes and methods. Tools are integrated which means information created by one tool can be used by another.

Tools help developers perform tasks efficiently.

Examples

- .Visual Studio

- .Eclipse

- .IntelliJ IDEA

- .Git

- .Jenkins

- .Selenium

 - Benefits

 - .Faster development

 - .Better testing

 - .Improved productivity

Advantages of Layered

Technology . Better software quality

. Eases project management .

Improves teamwork

. Reduces development time .

Increases customer satisfaction

A process framework

A **Software Process Framework** is a structured approach that defines the steps, tasks, and activities involved in software development. This framework serves as a

foundation for software engineering

Software Process

Process Frameworks

Umbrella Activities

Action #1.l
Task sets

Work Task
Milestone
SQA Points

Action #1.k
Task sets

Work Task
Milestone
SQA Points

Action #1.n
Task sets

Work Task
Milestone
SQA Points

Software Process Framework Activities

The five generic process framework activities useful in developing several projects are given below:

Software Process Framework Activities



1. Communication

Definition: Communication involves gathering requirements from customers and stakeholders to determine the system's objectives and the software's requirements. **Activities:**

- **Requirement Gathering:** Engaging with consumers and stakeholders through meetings, interviews, and surveys to understand their needs and expectations.
- **Objective Setting:** Clearly defining what the system should achieve based on the gathered requirements.

Explanation: Effective communication is essential to understand what the users need from the software. This phase ensures that all stakeholders are on the same page regarding the goals and requirements of the system.

2. Planning

Definition: Planning involves establishing an engineering work plan, describing technical risks, listing resource requirements, and defining a work schedule.

Activities:

- . **Work Plan:** Creating a detailed plan that outlines the tasks and activities needed to develop the software.
- . **Risk Assessment:** Identifying potential technical risks and planning how to mitigate them.
- . **Resource Allocation:** Determining the resources (time, personnel, tools) required for the project.
- . **Schedule Definition:** Setting a timeline for completing different phases of the project.

Explanation: Planning helps in organizing the project and setting clear expectations. It ensures that the development team has a roadmap to follow and that potential challenges are anticipated and managed.

Modeling

Definition: Modeling involves creating architectural models and designs to better understand the problem and work towards the best solution.

Activities:

- **Analysis of Requirements:** Breaking down the gathered requirements to understand what the system needs to do.
- **Design:** Creating architectural and detailed designs that outline how the software will be structured and how it will function.

Explanation: Modeling translates requirements into a visual and structured representation of the system. It helps in identifying the best design approach and serves as a blueprint for development.

4. Construction

Definition: Construction involves creating code, testing the system, fixing bugs, and confirming that all criteria are met.

Activities:

.**Code Generation:** Writing the actual code based on the design models. .**Testing:** Running tests to ensure the software works as intended, identifying and fixing bugs.

Explanation: This phase is where the actual software is built. Testing is crucial to ensure that the code is error-free and that the software meets all specified requirements.

5. Deployment

Definition: Deployment involves presenting the completed or partially completed product to customers for evaluation and feedback, then making necessary modifications based on their input.

Activities:

- **Product Release:** Delivering the software to users, either as a full release or in stages.
- **Feedback Collection:** Gathering feedback from users about their experience with the software.
- **Product Improvement:** Making changes and improvements based on user feedback to enhance the product.

Popular Software Development

Frameworks .Angular

.React

.Vue.js

.Django

.Flask

.Ruby on Rails

.Spring

.Express

.Laravel

.ASP.NET Core

Umbrella Activities

[Umbrella Activities](#) that take place during a [software development process](#) for improved project management and tracking.

- 1. Software project tracking and control:** This is an activity in which the team can assess progress and take corrective action to maintain the schedule. Take action to keep the project on time by comparing the project's progress against the plan.
- 2. Risk management:** The risks that may affect project outcomes or quality can be analyzed. Analyze potential risks that may have an impact on the software product's quality and outcome.
- 3. Software quality assurance:** These are activities required to maintain software quality.

Perform actions to ensure the product's quality.

- 4. Formal technical reviews:** It is required to assess engineering work products to uncover and remove errors before they propagate to the next activity. At each level of the process, errors are evaluated and fixed.
- 5. Software configuration management:** Managing of configuration process when any change in the software occurs.
- 6. Work product preparation and production:** The activities to create models, documents, logs, forms, and lists are carried out.
- 7. Reusability management:** It defines criteria for work product reuse. Reusable work items should be backed up, and reusable software components should be achieved.
- 8. Measurement:** In this activity, the process can be defined and collected. Also, project

and product measures are used to assist the software team in delivering the required software.

Process models

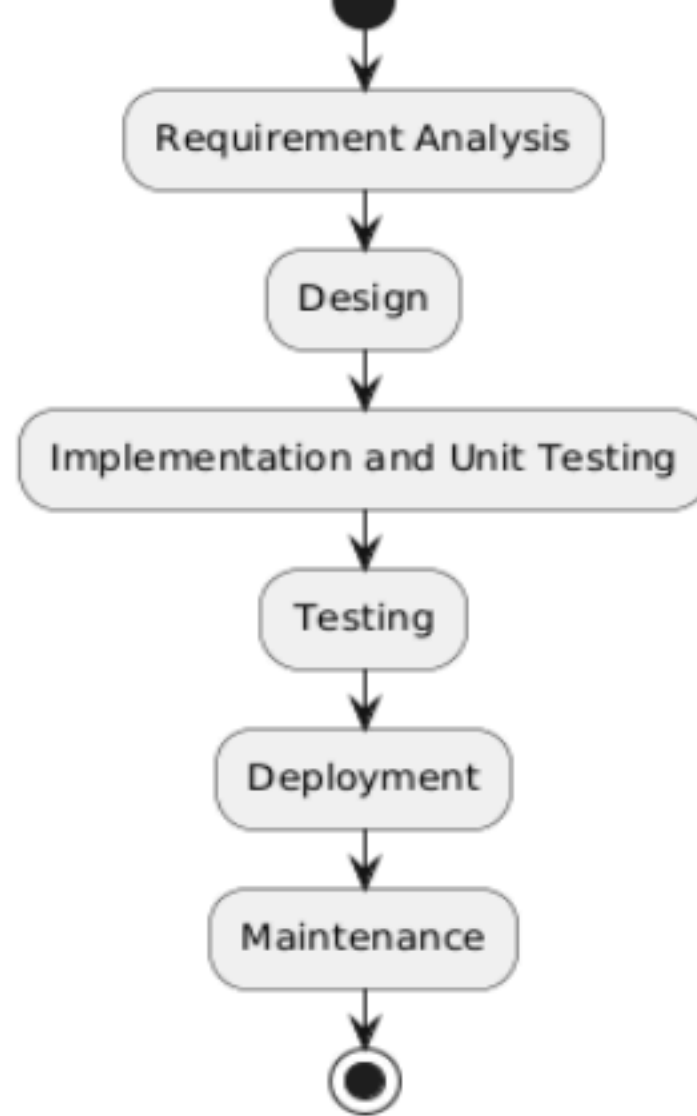
Software development life cycle

Software development life cycle (SDLC) is a structured process that is used to design, develop, and test good-quality software. SDLC, or software development life cycle, is a methodology that defines the entire procedure of software development step-by-step. The goal of the SDLC life cycle model is to deliver high-quality, maintainable software that meets the user's requirements. SDLC in software engineering models outlines the plan for each stage so that each stage of the software development model can perform its task efficiently to deliver the software at a low cost within a given time frame that meets user's requirements.

The SDLC Model involves six phases or stages while developing any software.

Requirements analysis phase

Requirements analysis



and specification phase is very important in the successful development of a software. If the customer requirements are not analysed properly, the chances of project failure or amount of rework on project becomes very high. Therefore, the project startup without proper requirements analysis and documentation is considered as a major mistake.

It also increases the burden on developer to reflect/implement the changes multiple times. Such iterations increase the cost incurred in the overall project and also leads to customer dissatisfaction.

The importance of requirements understanding and documentation is independent of the type of project i.e., internal or contract based. The documentation generated after successful analysis and specification is called Software Requirement Specification (SRS) document. This document not only helps in getting a clear

understanding about product requirements, but also helps in carrying out different activities of later phases.

Design phase

Design is a meaningful engineering representation of something that is to be built. It can be traced to a customer's requirements and at the same time assessed for quality against a set of predefined criteria for “good” design. In the software engineering context, design focuses on four major areas of concern – data, architecture, interfaces and components.

Software design process is an iterative process through which requirements document (SRS) is transformed to design document.

Implementation and unit testing phase

In this phase, actual code is developed using a programming language. As soon as the

code is developed, each unit or component in it is tested individually. Such testing is called unit testing. It ensures that maximum errors are detected and entire code is tested by following certain paths in the control structure

Testing phase

Testing is a major activity for software development and its main function is to detect bugs or errors that occur while executing a software. During the phases like requirement analysis and system design, the output results in the form of textual documents whereas, for the coding phase, the output is a computer program. Hence, testing not only detect errors during

Deployment phase

This forms the final phase of the software development process where the software is usually delivered to the end users for its effective implementation. Later, feedback is

collected from the users and if, the software fails to meet the users requirements, it is modified. Hence, the important activities involved in this phase are:-

- .Perfect delivery
- .Support and

- .Feedbacks.

Maintenance phase

Software maintenance refers to the changes which are performed once the software is delivered to users. Maintenance activity is compulsory for each type of product and cannot be avoided. Generally, products require maintenance activity due to its heavy usage. But, software need maintenance in order to improve features, correct errors. It is an essential activity adopted by large number of organization because of the misuse of hardware.

Top 8 Software Development Models

1. Waterfall Model
 2. Incremental Model
 3. Agile Model
 4. V-Model
 5. RAD Model
 6. Iterative Model
 7. Spiral Model
 8. Prototype model
- Waterfall Model

The Waterfall Model was the first Process Model to be introduced. It is also referred to as a **linear-sequential life cycle**

model.

It is very simple to understand and use.

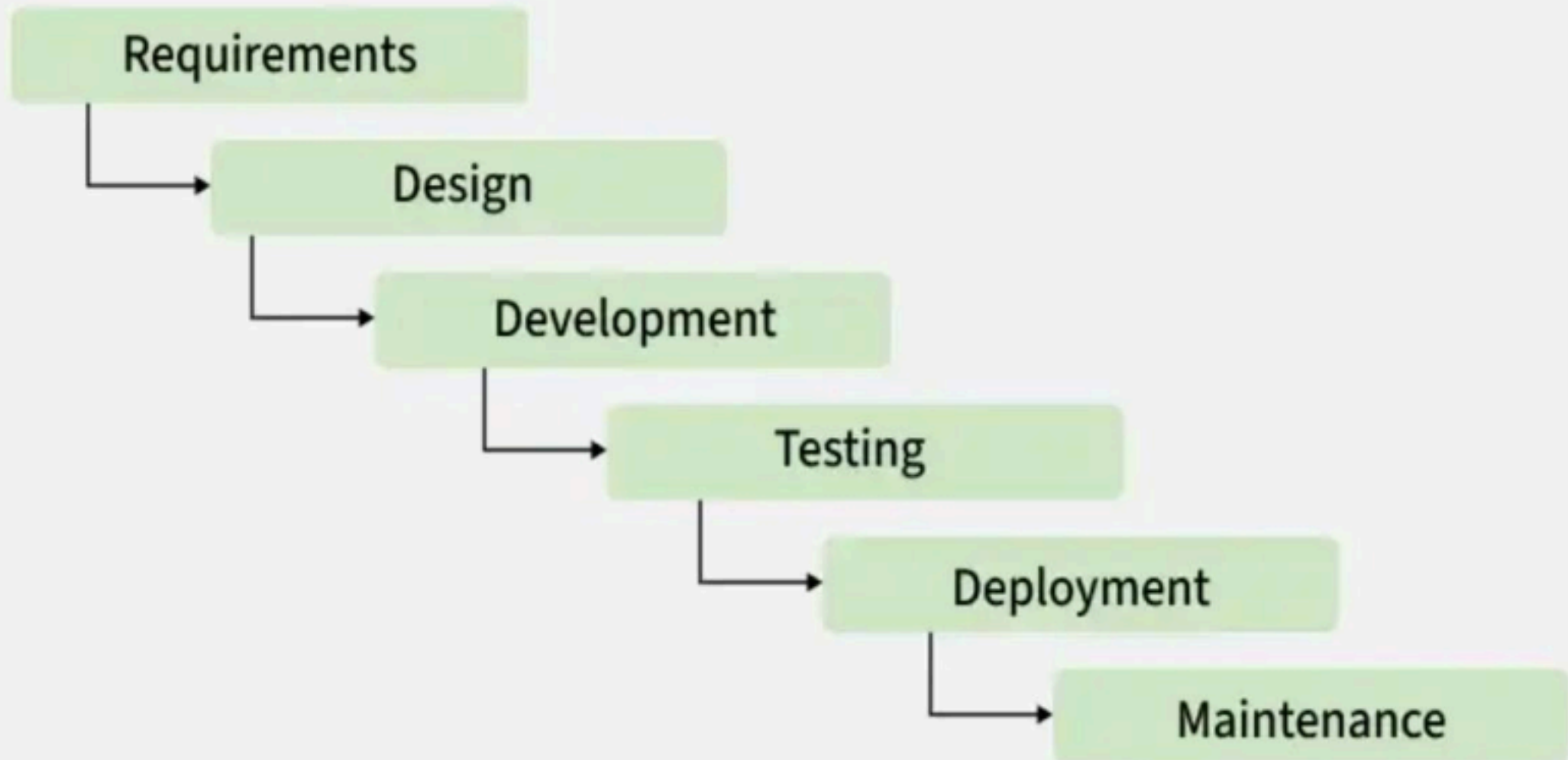
In a waterfall model, each phase must be completed before the next phase can begin and there is no overlapping in the phases. The Waterfall model is the earliest SDLC approach that was used for software development.

Waterfall approach to be used widely in Software Engineering to ensure success of the project.

In "The Waterfall" approach, the whole process of software development is divided into separate phases.

In this Waterfall model, typically, the outcome of one phase acts as the input for the next phase sequentially.

Phases of WaterFall Model



Requirement Gathering and analysis –The first stage focuses on clearly understanding and documenting the customer's needs.

Requirement Gathering and Analysis: Customer requirements are collected and carefully examined to remove errors, confusion, and inconsistencies.

Requirement Specification: The approved requirements are documented in the [Software Requirement Specification \(SRS\)](#), which serves as a formal agreement between the customer and the development team.

System Design – The requirement specifications from first phase are studied in this phase and the system design is prepared. This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture. The requirements specified in the SRS are converted into a system design that can be implemented in code.

High-Level Design (HLD): Defines the overall system architecture, major components, and their interactions.

Low-Level Design (LLD): Provides detailed design of each component, including logic and data flow, to guide developers during coding.

All designs are documented in the [Software Design Document \(SDD\)](#).

Development – This phase involves converting the design into actual working software. Developers write source code based on the design documents.

Suitable programming languages and tools are used.

[Unit testing](#) is performed to verify that each module works correctly on its own.

Testing: After unit testing, software modules are integrated and integration testing is performed. Once integration is complete, system testing is carried out to verify the entire system.

Alpha Testing: Performed by the development team.

Beta Testing: Performed by selected end users.

Acceptance Testing: Performed by the customer to approve the software.

Deployment of system – After successful testing, the software is deployed to a live environment for end users. This phase includes environment setup, user training, and final checks to ensure smooth operation in real world conditions.

Maintenance

Maintenance ensures the software continues to function effectively after deployment. • Corrective Maintenance: Fixes errors found after release.

• Perfective Maintenance: Enhances features based on user needs. •

Adaptive Maintenance: Adapts software to new environments. •

Preventive Maintenance: Prevents future issues through improvements.

Advantages



- A waterfall model is easy to implement.
- It helps to find errors earlier.
- Easy to understand, and easy to use.
- Testing is done at every stage.
- Works well when quality is more important than cost or schedule.
- Documentation is produced at every stage of a waterfall model allowing people to understand what has been done.

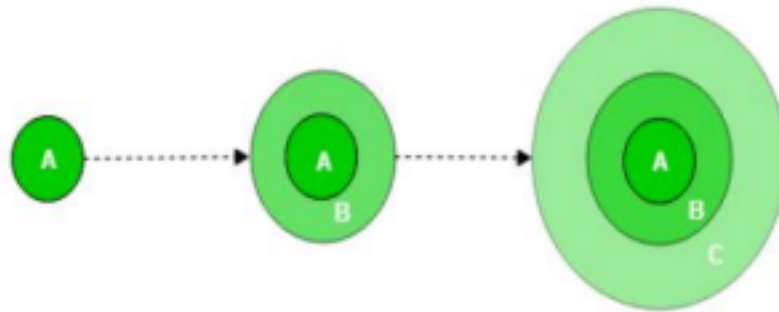
Application of the Waterfall model

- Projects are large, complex, and require a well-structured, sequential, linear approach. This helps projects to be developed on time and within budget.
- Safety-critical systems such as aerospace and medical systems.
- Government projects, defense – security project.
- Projects with clear and stable requirements documentation.

Incremental process model

Incremental process model is also known as Successive version model.

First, a simple working system implementing only a few basic features is built and then that is delivered to the customer. Then thereafter many successive iterations/ versions are implemented and delivered to the customer until the desired system is released.



A, B, C are modules of Software Product that are incrementally developed and delivered.

Incremental Model is a process of software development where requirements

divided into multiple standalone modules of the software development cycle.

In this model, each module goes through the requirements, design, implementation and testing phases. Every subsequent release of the module adds function to the previous release.

The process continues until the complete system achieved.

The various phases of incremental model are as follows:

Requirement analysis: In the first phase of the incremental model, the product analysis expertise identifies the requirements. And the system functional requirements are understood by the requirement analysis team. To develop

the software under the incremental model, this phase performs a crucial role.

Design & Development: In this phase of the Incremental model of SDLC, the design of the system functionality and the development method are finished with success. When software develops new practicality, the incremental model uses style and development phase.

Testing: In the incremental model, the testing phase checks the performance of each existing function as well as additional functionality. In the testing phase, the various methods are used to test the behavior of each task.

Implementation: Implementation phase enables the coding phase of the development system. It involves the final coding that design in the designing and development phase and tests the functionality in the testing phase. After completion of this phase, the number of the product working is enhanced and upgraded up to the final system product

When we use the Incremental Model?

- When the requirements are superior.

- A project has a lengthy development schedule.
- When Software team are not very well skilled or trained.
- When the customer demands a quick release of the product.
- You can develop prioritized requirements first.

Advantage of Incremental Model

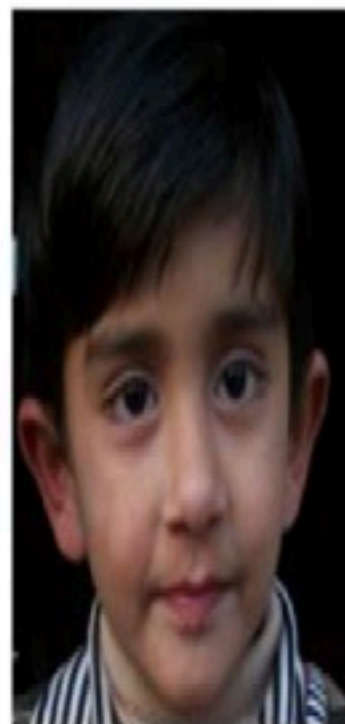
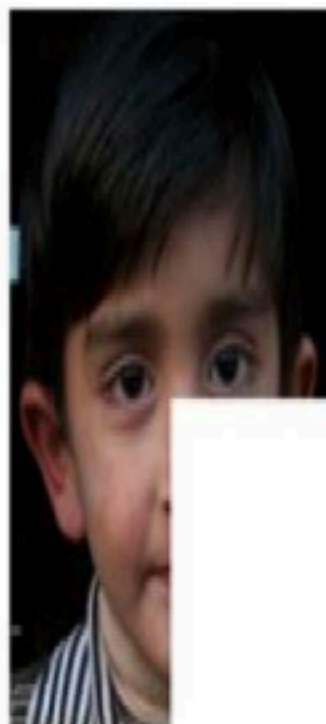
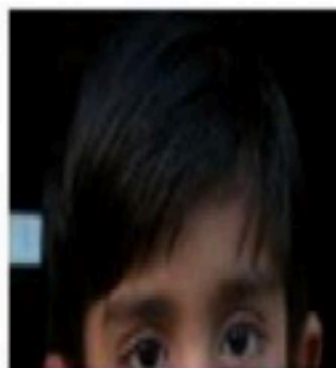
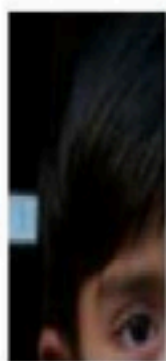
- Errors are easy to be recognized.
- Easier to test and debug

- More flexible.
- Simple to manage risk because it handled during its iteration.
- The Client gets important functionality early.

Disadvantage of Incremental Model

- Need for good planning
- Total Cost is high.
- Well defined module interfaces are needed.

Example



www.t4tutorials.com

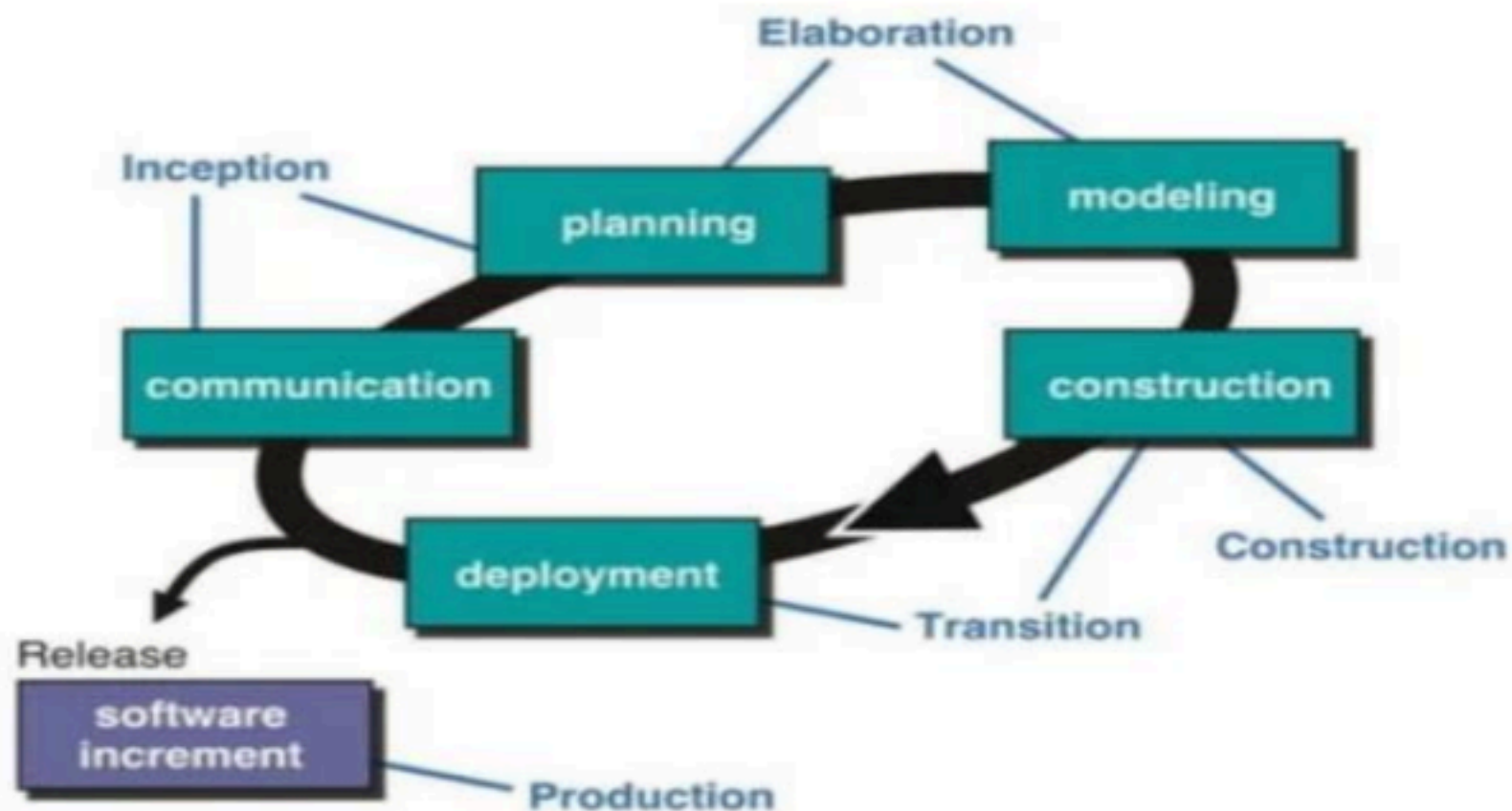
Increment 1

Increment 2

Increment 3

Increment 4

Unified Process Model



The **Unified Process (UP) Model** is a **software development methodology** in which a system is developed **iteratively and incrementally**. Instead of developing the complete software in one go, the project is divided into several **iterations (cycles)**. Each iteration involves activities such as **requirements analysis, design, coding, and testing**, producing an improved version of the software.

1. Iterative and Incremental

he software is developed in repeated cycles. Each cycle adds new functionality or improves existing functionality. **Example:**

Iteration 1 → Login

Iteration 2 → Account management

Iteration 3 → Money transfer

Iteration 4 → Reports

. **Use-Case Driven**

Development is based on **use cases**, which describe how users interact with the system. For example, in an ATM:

Withdraw cash

Check balance

Deposit money

Change PIN

3. Architecture-Centric

The Unified Process gives importance to developing a strong **software architecture** early in the project. This helps ensure that the system can support future requirements.

4. Risk-Focused

Important technical and business risks are identified and addressed early, particularly during the **Elaboration phase**

Phases of Unified Process

UNIFIED PROCESS MODEL



Inception Phase

The inception phase has the following objectives:

- Gathering and analyzing the requirements.
- Planning and preparing a business case and evaluating alternatives for risk management, scheduling resources etc.
- Estimating the overall cost and schedule for the project.
- Studying the feasibility and calculating profitability of the project.

Elaboration Phase

The elaboration phase has the following objectives:

- Establishing architectural foundations.
- Design of use case model.
- Elaborating the process, infrastructure & development environment.
- Selecting component.
- Demonstrating that architecture support the vision at reasonable cost & within specified time.

Construction Phase

The construction phase has the following objectives:

- Implementing the project.
- Minimizing development cost.
- Management and optimizing resources.
- Testing the product
- Assessing the product releases against acceptance criteria

Transition Phase

The transition phase has the following objectives:

- Starting of beta testing
- Analysis of user's views.
- Training of users.
- Tuning activities including bug fixing and enhancements for performance & usability
- Assessing the customer satisfaction.



Main Workflow Activities

Phase Main Activities

Inception Requirements, scope, feasibility **Elaboration** Analysis, risk identification, architecture **Construction** Design, coding, testing, integration

Transition Deployment, user feedback, maintenance

- **Advantages of the Unified Process Model**

- **Iterative development** – The software is developed in small, manageable iterations.
- **Early risk detection** – Major risks are identified and addressed early.
- **Supports changing requirements** – Changes in requirements can be accommodated easily.
- **Continuous testing** – Testing is performed throughout the development process.
- **Customer feedback** – Regular feedback from customers helps improve the final

product.

Disadvantages of Unified Process Model

- 1. Complex to manage** – It can be difficult to understand and manage due to its many activities and phases.
- 2. Costly** – Requires more resources, documentation, and skilled professionals.
- 3. Time-consuming** – Continuous iterations, testing, and documentation can increase development time.
- 4. Requires experienced team members** – Proper implementation needs skilled

and experienced developers.

5. Not suitable for small projects – Its detailed process can be excessive for simple or small software projects.

Applications of Unified Process Model

1. Banking software – For developing secure and complex banking systems.

2. E-commerce systems – For online shopping and payment applications. **3.**

Healthcare systems – For hospital and patient management software. **4.**

Enterprise applications – For ERP, CRM, and other business systems. **5.**

Telecommunication systems – For developing complex communication and

network software.

Agile Software Development

Agile Software Development is an approach to software development in which software is developed **incrementally and iteratively**, with continuous customer feedback and the ability to respond quickly to changing requirements. Traditional development methods often try to define most requirements at the beginning. Agile accepts that requirements may change and therefore emphasizes **flexibility, collaboration, and frequent delivery of working software**.

Main Characteristics

- Development is divided into **small iterations**.
- Working software is delivered frequently.
- Customer feedback is obtained continuously.
- Changing requirements are accepted.

- Developers, customers, and other stakeholders collaborate closely.
- Testing occurs throughout development.
- Teams continuously improve their development process.

Agile Development Cycle

Planning → Design → Coding → Testing → Working Software → Customer Feedback → Next

Iteration The cycle is repeated until the required system is completed.



12 Principles of Agile

1. Customer satisfaction

Deliver valuable software **early and continuously** to satisfy the customer.

2. Welcome changing requirements

Changes in requirements are welcomed, even late in development.

3. Frequent delivery

Deliver working software frequently, preferably in short time intervals.

4. Business and developers work together

Customers, business people, and developers should collaborate throughout the project.

5. Motivated individuals

Build projects around motivated people and provide them with the necessary support and environment.

6. Effective communication

Face-to-face communication is considered an effective way of communicating information.

7. Working software

Working software is the primary measure of progress.

8. Sustainable development

The team should maintain a sustainable pace that can be continued over the long term.

9. Technical excellence

Continuous attention to good design, technical quality, and engineering practices improves agility. **10. Simplicity**

Maximize the amount of work **not done**. Avoid unnecessary features and activities. **11. Self-organizing teams**

The best architectures, requirements, and designs emerge from self-organizing teams.

12. Continuous improvement

At regular intervals, the team reflects on its performance and adjusts its behavior to improve.

Agile Methods

Agile methods are software development methodologies that follow Agile principles and values.

Common Agile methods include:

Extreme Programming (XP)

Scrum

Kanban

Crystal

Dynamic Systems Development Method (DSDM)

Adaptive Software Development (ASD)

Feature-Driven Development (FDD)

Among these, **Extreme Programming and Scrum** are particularly important.

Advantages of Agile

1. Flexibility

Agile can respond to changing customer requirements.

2. Early Delivery

Customers receive useful software early.

3. Continuous Feedback

Problems and misunderstandings can be identified quickly. **4. Better Quality**

Frequent testing and integration help detect defects early. **5. Reduced Risk**

Risks are identified and addressed throughout development. **6. Customer Satisfaction**

Customers are continuously involved and can see progress. **7. Improved Teamwork**

Agile encourages communication and collaboration.